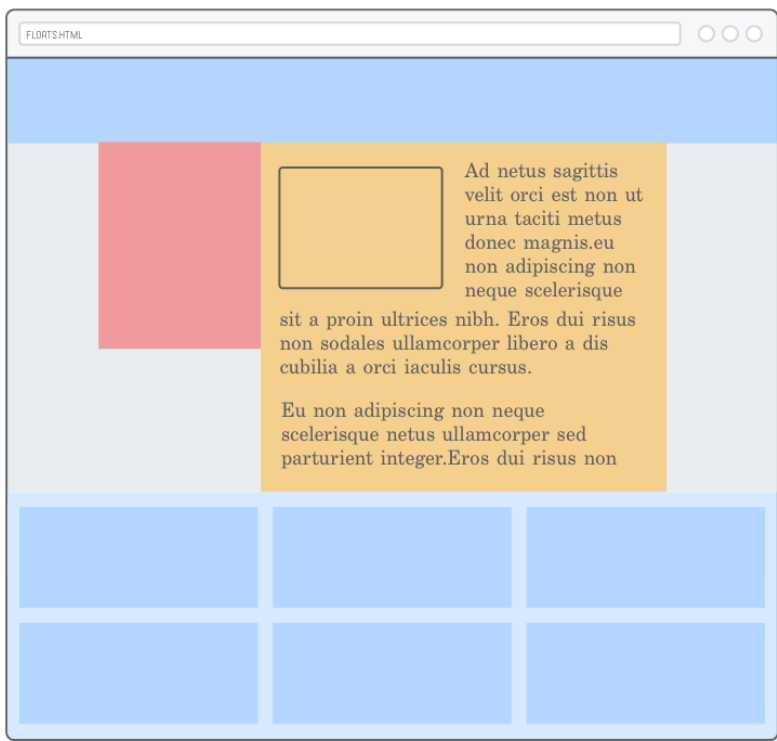


接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

CSS布局概述

- **CSS页面布局过程：**选择网页中的元素，并且控制它们相对正常文档流、周边元素、父容器或者主视口/窗口的位置



正常文档流

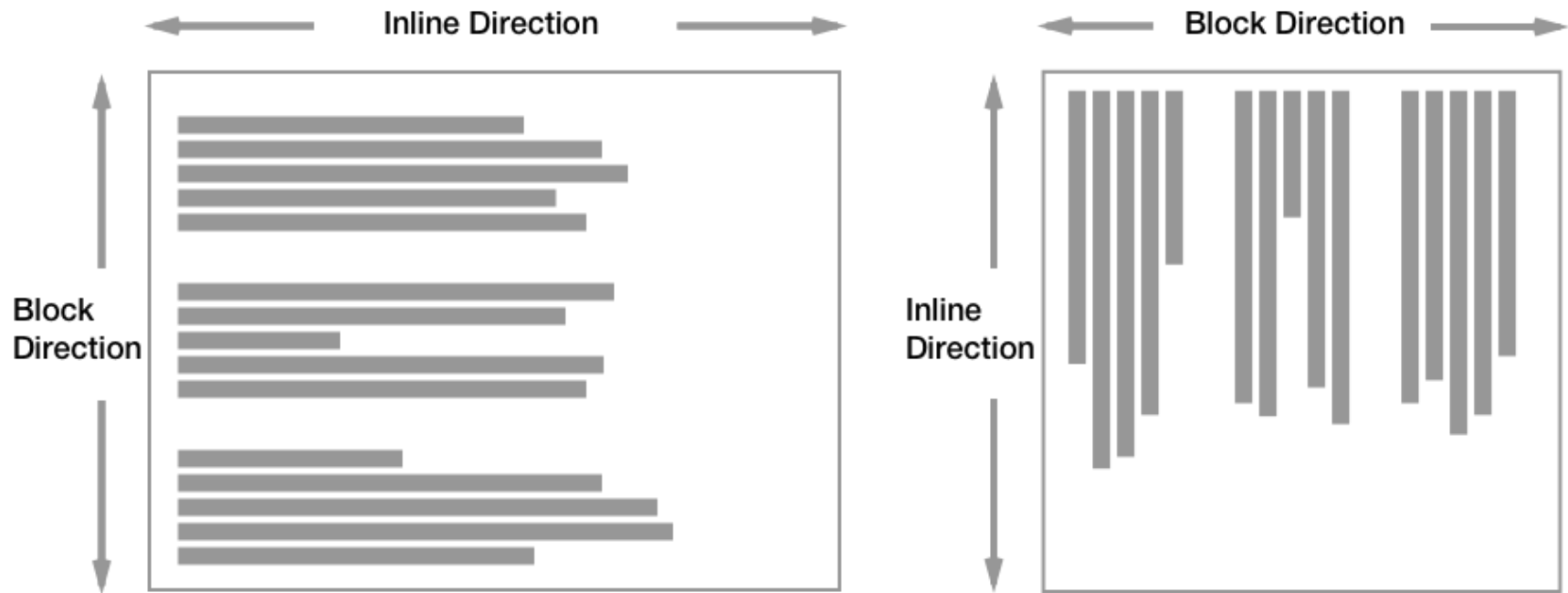


脱离正常文档流

在CSS布局中，可以通过设置某些属性实现“脱离正常文档流”，以控制页面布局

正常文档流

- 文档流是元素在页面出现的先后顺序，如果没用任何CSS来改变页面布局，HTML元素就会排列在**正常文档流（Normal Flow）**之中



在**正常文档流**之中，元素盒子（任何一个HTML元素其实就是一个盒子）会基于文档的**书写顺序**一个接一个地排列，将窗体自上而下分成一行一行，块元素独占一行，相邻行内元素在每行中按从左到右地依次排列元素

正常文档流

- 通过页面结构来发挥正常文档流的优势
 - 确保书写的页面具有良好的语义结构，最大程度复用正常文档流所带来的优势
 - 正常文档流避免指定所有的HTML元素的布局方式，即使CSS加载失败了，用户仍然能阅读网页内容
 - 确保不使用CSS的工具（例如屏幕阅读器）会按照元素在文档中的位置来读取页面内容
 - 如果网页内容顺序和用户预期的阅读顺序一致，那就不需要为了将元素调整到正确的位置而做大量的布局调整
 - 在思考如何布局之前，需要认真思考文档结构，以及希望用户以何种顺序来阅读文档中的内容

浮动float

- float允许元素浮动到另外一个元素的左侧或右侧，而不是默认的一个堆叠另一个
 - left — 将元素浮动到左侧
 - right — 将元素浮动到右侧
 - none — 默认值, 不浮动
 - inherit — 继承父元素的浮动属性
- float 的主要用途是布置出多个列并且浮动文字以环绕图片

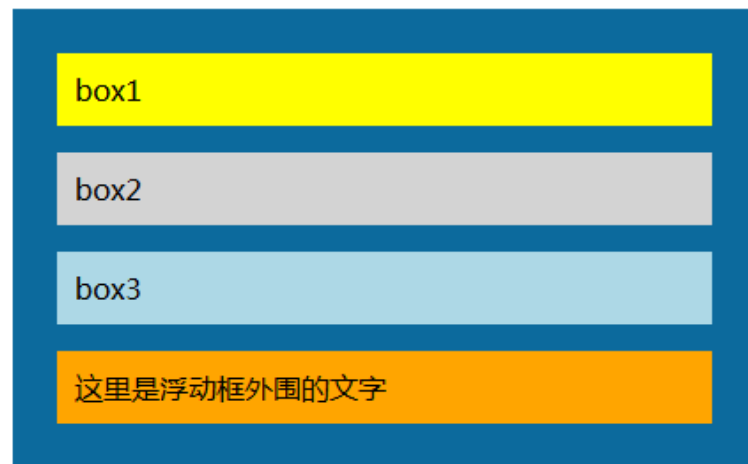
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9     #son1 { background-color:yellow; }
10    #son2 { background-color:lightgrey; }
11    #son3 {background-color:lightblue; }
12  </style>
13 </head>
14 <body>
15   <div id="father">
16     <div id="son1">box1</div>
17     <div id="son2">box2</div>
18     <div id="son3">box3</div>
19     <p>这里是浮动框外围的文字</p>
20   </div>
21 </body>
22 </html>
```

显示效果



定义了4个div块，一个是父块，另外3个是它的子块。3个子块都没有设置浮动方式，在父盒子中，由于div元素是块元素，因此4个盒子各自向右伸展，并且自上而下排列

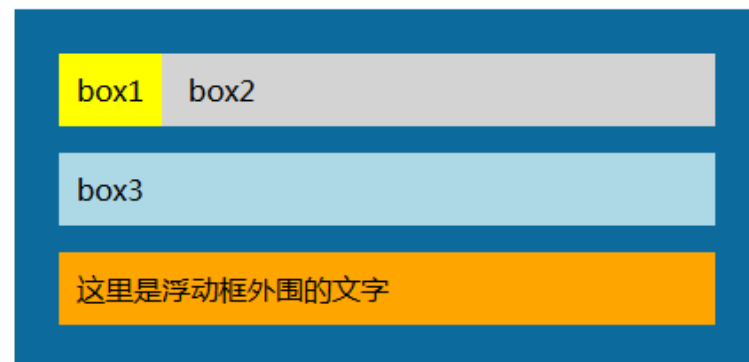
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9
10    #son1 { float:left; background-color:yellow; }
11
12    #son2 { background-color:lightgrey; }
13    #son3 {background-color:lightblue; }
14  </style>
15 </head>
16 <body>
17   <div id="father">
18     <div id="son1">box1</div>
19     <div id="son2">box2</div>
20     <div id="son3">box3</div>
21     <p>这里是浮动框外围的文字</p>
22   </div>
23 </body>
24 </html>
```

显示效果



由于box1设置为左浮动，box1变成了浮动元素，因此此时box1的宽度不再延伸，其宽度为容纳内容的最小宽度，而相邻的下一个div元素（box2）就会紧贴着box1，这是由于浮动引起的效果

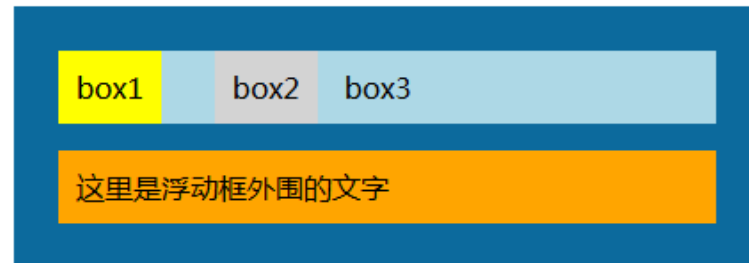
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9
10    #son1 { float:left; background-color:yellow; }
11    #son2 { float:left; background-color:lightgrey; }
12
13    #son3 {background-color:lightblue; }
14  </style>
15 </head>
16 <body>
17   <div id="father">
18     <div id="son1">box1</div>
19     <div id="son2">box2</div>
20     <div id="son3">box3</div>
21     <p>这里是浮动框外围的文字</p>
22   </div>
23 </body>
24 </html>
```

显示效果



由于box2变成了浮动元素，因此box2也跟box1一样，宽度不再延伸，而是由内容确定宽度。并且相邻的下一个div元素（box3）变成紧贴着浮动的box2。如果box1为浮动元素，而相邻的box2不是浮动元素，则box2就会紧贴着box1，外边距不生效；**但是如果box1和box2同时为浮动元素，外边距就会生效**

浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9
10    #son1 { float:left; background-color:yellow; }
11    #son2 { float:right; background-color:lightgrey; }
12
13    #son3 {background-color:lightblue; }
14  </style>
15 </head>
16 <body>
17   <div id="father">
18     <div id="son1">box1</div>
19     <div id="son2">box2</div>
20     <div id="son3">box3</div>
21     <p>这里是浮动框外围的文字</p>
22   </div>
23 </body>
24 </html>
```

显示效果



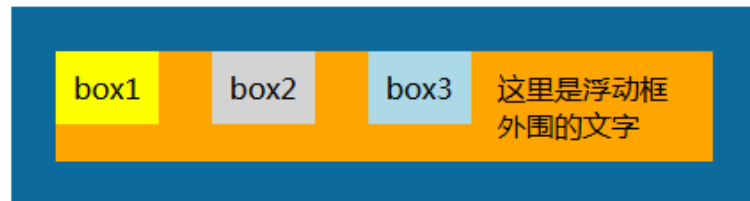
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9     #son1 { float:left; background-color:yellow; }
10    #son2 { float:left; background-color:lightgrey; }
11    #son3 {float:left; background-color:lightblue; }
12  </style>
13 </head>
14 <body>
15   <div id="father">
16     <div id="son1">box1</div>
17     <div id="son2">box2</div>
18     <div id="son3">box3</div>
19     <p>这里是浮动框外围的文字</p>
20   </div>
21 </body>
22 </html>
```

显示效果



请仔细观察哪些margin生效了？

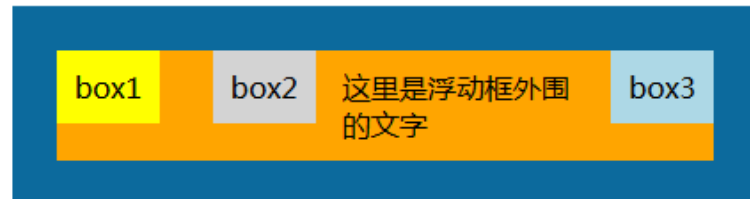
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father { width:400px; background-color:#0C6A9D;}
7     div {padding:10px; margin:15px;}
8     p { padding:10px; margin:15px; background-color:orange; }
9     #son1 { float:left; background-color:yellow; }
10    #son2 { float:left; background-color:lightgrey; }
11    #son3 {float:right; background-color:lightblue; }
12  </style>
13 </head>
14 <body>
15   <div id="father">
16     <div id="son1">box1</div>
17     <div id="son2">box2</div>
18     <div id="son3">box3</div>
19     <p>这里是浮动框外围的文字</p>
20   </div>
21 </body>
22 </html>
```

显示效果



请仔细观察哪些margin生效了？

浮动float

- 清除浮动

- 一旦对一个元素进行了浮动，所有接下来的元素都会环绕它
- 如果不想要在某个元素受到其之前的浮动元素影响时，可以为其添加clear属性清除浮动，接下来的部分会以正常流方式绘制在其后
 - clear:取值;

表1 clear属性值

clear属性值	说明
left	清除左浮动
right	清除右浮动
both	左右浮动一起清除

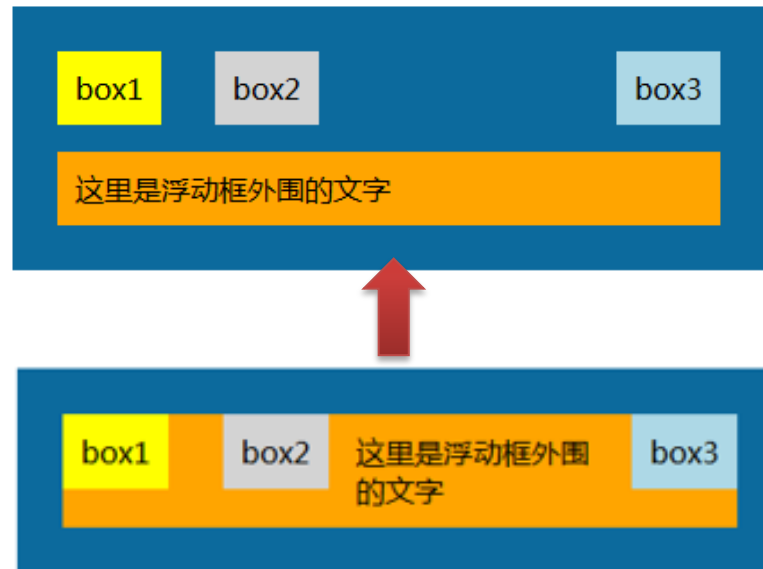
浮动float

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4 <title></title>
5 <style type="text/css">
6 #father { width:400px; background-color:#0C6A9D;}
7 div {padding:10px; margin:15px;}
8 p { padding:10px; margin:15px; background-color:orange; }
9 #son1 { float:left; background-color:yellow; }
10 #son2 { float:left; background-color:lightgrey; }
11 #son3 { float:right; background-color:lightblue; }
12 p { clear:both;}
13 </style>
14 </head>
15 <body>
16 <div id="father">
17 <div id="son1">box1</div>
18 <div id="son2">box2</div>
19 <div id="son3">box3</div>
20 <p>这里是浮动框外围的文字</p>
21 </div>
22 </body>
23 </html>
```

显示效果



由于p元素清除了浮动，所以p元素的前一个元素产生的浮动就不会对后续元素产生影响，因此p元素的文本不会环绕在浮动元素的周围

浮动float

- 用浮动来创建一个简单的两列布局

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4 <title></title>
5 <style type="text/css">
6   div:nth-of-type(1) {
7     width: 48%;
8     float: left; }
9   div:nth-of-type(2) {
10    width: 48%;
11    float: right; }
12 </style>
13 </head>
14 <body>
15   <h1>双栏布局案例</h1>
16   <div>
17     <h2>第一栏</h2>
18     <p>这是第一栏的内容这是第一栏的内容这是第一栏的内容这是第一栏的内容</p>
19   </div>
20   <div>
21     <h2>第二栏</h2>
22     <p>这是第二栏的内容这是第二栏的内容这是第二栏的内容这是第二栏的内容</p>
23   </div>
24 </body>
25 </html>
```

显示效果

双栏布局案例

第一栏

这是第一栏的内容这是第一栏的内容这是第一栏的内容这是第一栏的内容

第二栏

这是第二栏的内容这是第二栏的内容这是第二栏的内容这是第二栏的内容

将两个 <div>元素排成一行，两个 <div>一个浮动值为 left 往左靠，另外一个浮动为 right 往右靠，这两个元素分别设置 width 值，使得它们能够在同一行放下来，注意水平间距总宽度不要大于100%!

浮动float

- 利用float属性定义文字环绕效果【回顾】

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5   <style type="text/css">
6     img{margin-right:20px;margin-bottom:20px;float:left;}
7   </style>
8 </head>
9 <body>
10   
11   <p>水陆草木之花，可爱者甚蕃。晋陶渊明独爱菊。自李唐来，世人甚爱牡丹。予
12 </body>
13 </html>
```

显示效果



水陆草木之花，可爱者甚蕃。晋陶渊明独爱菊。自李唐来，世人甚爱牡丹。予独爱莲之出淤泥而不染，濯清涟而不妖，中通外直，不蔓不枝，香远益清，亭亭净植，可远观而不可亵玩焉。予谓菊，花之隐逸者也；牡丹，花之富贵者也；莲，花之君子者也。噫！菊之爱，陶后鲜有闻；莲之爱，同予者何人？牡丹之爱，宜乎众矣。

定位

- 定位布局简介

- 定位技术(position)允许我们将一个元素从它在页面的原始位置准确地移动到另外一个位置

- 静态定位(Static positioning)是每个元素默认的属性，表示将元素放在文档布局流的默认位置
 - 相对定位(Relative positioning)允许我们按照元素在正常文档流中的相对位置移动它
 - 绝对定位(Absolute positioning)将元素完全从页面的正常布局流中移出，将它单独放在一个图层中，设定相对与另一个元素的位置
 - 固定定位(Fixed positioning)与绝对定位类似，将元素相对浏览器视口固定，而不是相对另外一个元素

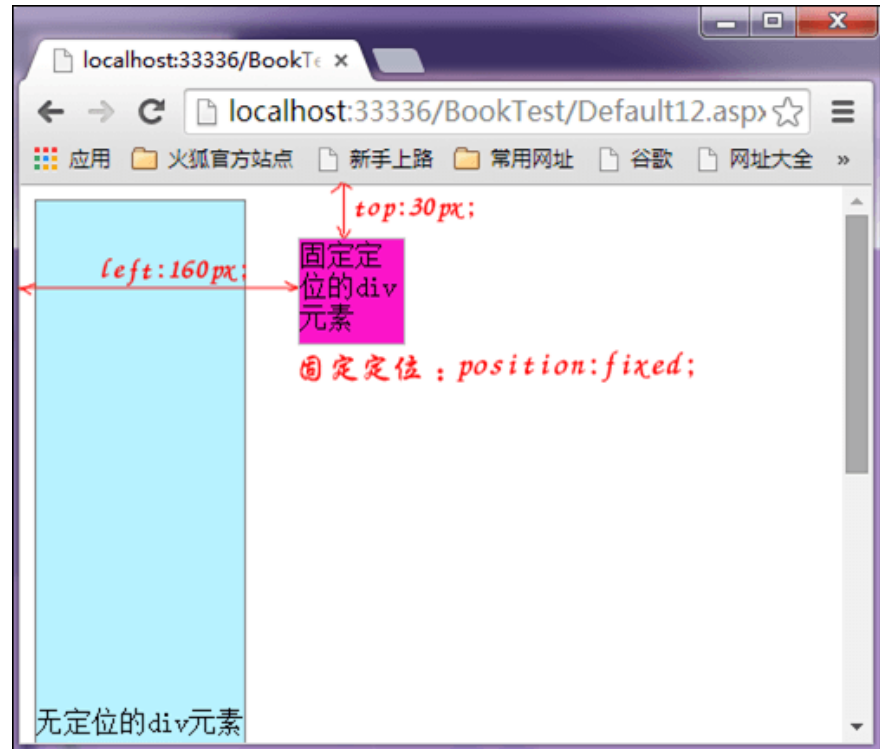
定位

- 固定定位

- 当元素的position属性设置为fixed时，这个元素就被固定了，被固定的元素不会随着滚动条的拖动而改变位置。在视野中，固定定位的元素的位置是不会改变的
- position:fixed;是结合top、bottom、left和right这4个属性一起使用的
 - position:fixed;使得元素成为固定定位元素
 - 使用top、bottom、left和right这4个属性来设置元素相对浏览器四个边的位置

在线代码测试

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #first {
7       width:120px;
8       height:600px;
9       border:1px solid gray;
10      line-height:600px;
11      background-color:#B7E1FF;
12    }
13    #second {
14      position:fixed; /*设置元素为固定定位*/
15      top:30px; /*距离浏览器顶部30px*/
16      left:160px; /*举例浏览器左部160px*/
17      width:60px;
18      height:60px;
19      border:1px solid silver;
20      background-color:#FA16C9;
21    }
22  </style>
23  <body>
24    <div id="first">无定位的div元素</div>
25    <div id="second">固定定位的div元素</div>
26  </body>
27 </html>
```



拖动浏览器的滚动条，固定定位的div元素不会有任何位置改变，但是无定位的div元素会改变

固定定位用途很多，一般用于“回顶部”特效和固定栏目的设置

定位

- 相对定位

- 采用相对定位的元素，其位置是相对于它的原始位置计算而来的，通过将元素从原来的位置向上、向下、向左或者向右移动来定位的

- `position:relative`;使元素成为相对定位元素

- 使用`top`、`bottom`、`left`和`right`这4个属性来设置元素相对原始位置的位移

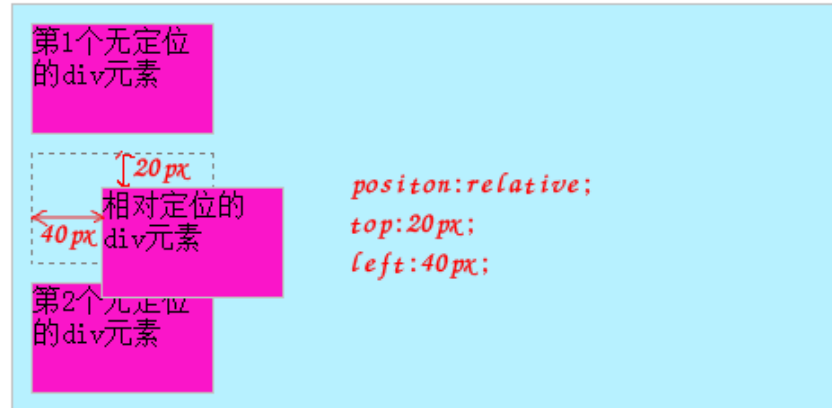
- 默认情况下，相对定位元素的位置是相对于原始位置而言，固定定位元素的位置是相对浏览器而言

定位

• 相对定位

在线代码测试

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father {
7       margin-top:30px; margin-left:30px;
8       border:1px solid silver;
9       background-color: #B7F1FF;    }
10    #father div {
11      width:100px; height:60px;
12      margin:10px;
13      border:1px solid silver;
14      background-color:#FA16C9;    }
15    #son2 {
16      position:relative;
17      top:20px; left:40px;    }
18  </style>
19 </head>
20 <body>
21   <div id="father">
22     <div id="son1">第1个无定位的div元素</div>
23     <div id="son2">相对定位的div元素</div>
24     <div id="son3">第2个无定位的div元素</div>
25   </div>
26 </body>
27 </html>
```



页面上的其他元素并不会因相对定位元素的位置变化而受到影响，该元素在正常文档流中的位置会被保留

定位

- 绝对定位

- 当元素的position属性值为absolute时，这个元素就变成了绝对定位元素，绝对定位能够很精确地把元素移动到任意位置
 - position:absolute;使得元素成为绝对定位元素
 - 使用top、bottom、left和right这4个属性来设置元素相对浏览器四条边的位置
- 默认情况下，绝对定位的元素的位置相对于最近的已定位父元素，如果元素没有已定位的父元素，那么它的位置相对于<html>，绝对定位的位置是相对原始位置而言

定位

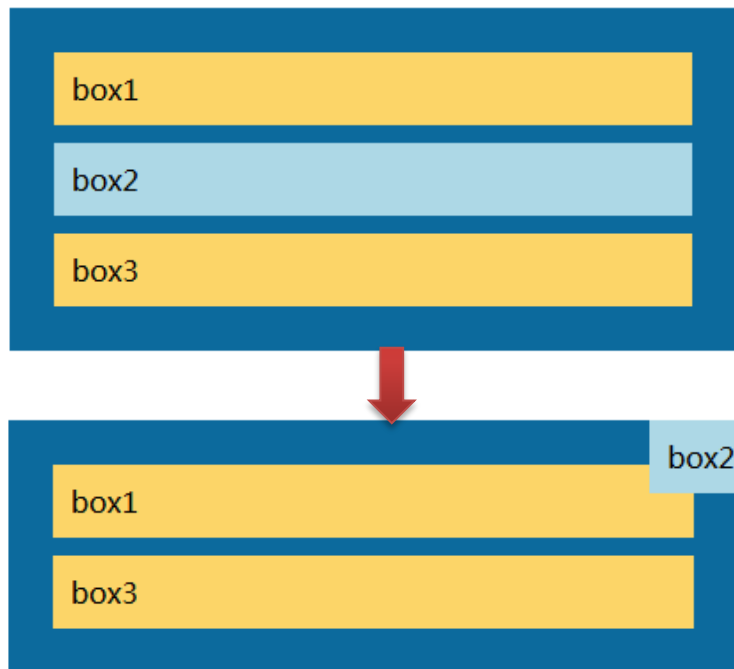
• 绝对定位

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father {
7       padding:15px; margin:10px; /*position: relative; */
8       background-color:#0C6A9D; }
9   #father div {
10    padding:10px; margin:10px;
11    background-color:#FCD568; }
12   #father #son2 {
13     background-color:lightblue;
14     /*position:absolute; top:0; right:0;*/
15   }
16 </style>
17 </head>
18 <body>
19   <div id="father">
20     <div id="son1">box1</div>
21     <div id="son2">box2</div>
22     <div id="son3">box3</div>
23   </div>
24 </body>
25 </html>
```

显示效果



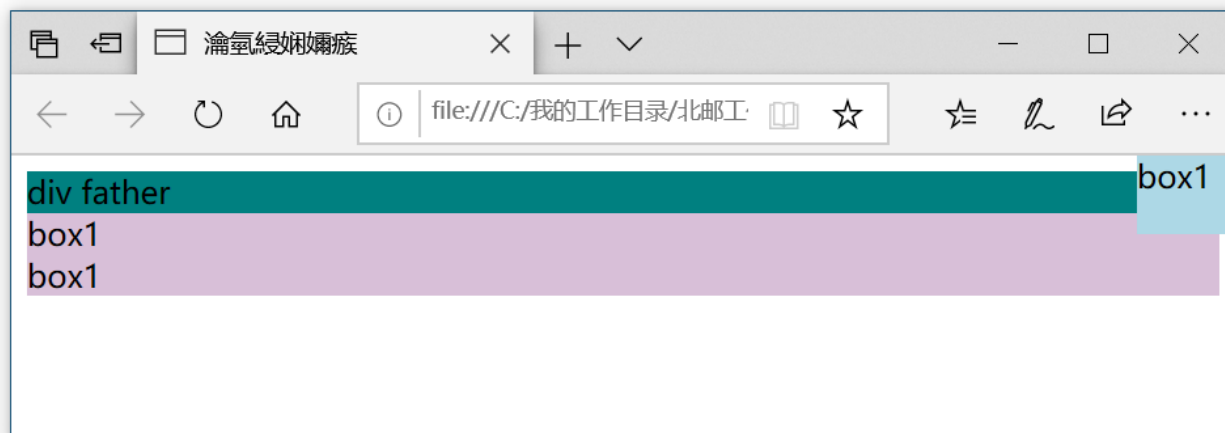
一个元素变成了绝对定位元素，这个元素就完全脱离正常文档流，绝对定位元素的前面或者后面的元素会认为这个元素并不存在，即这个元素浮于其他元素上面，它是独立出来的，**其原本占据的空间也会被移除**

定位

• 绝对定位

```
<!DOCTYPE html>
<html>
<head>
  <title>定位测试</title>
</head>
<body>
  <div id = 'father'>
    <div id = 'son1'>box1</div>
    <div id = 'son2'>box1</div>
    <div id = 'son3'>box1</div>
  </div>
</body>
</html>
```

```
<style>
  #father {padding: 0px; margin: 0px;
            background-color: teal;}
  #father div {padding: 0px; margin: 0px;
               background-color: thistle;}
  #father #son2 {background-color: lightblue; margin: 0px;
                 position: absolute; top: 0px; right: 0px; width: 50px; height: 40px;}
</style>
```



定位

- 静态定位

- 没有指定元素的position属性值，也就是默认情况下，元素是静态定位
- 任意 position: static; 的元素不会被特殊的定位
- 静态定位的元素按照文档流正常布局，并不受到其他定位方式的影响。
- 不受到 top、right、bottom、left 属性影响：使用静态定位的元素无法通过设置这些偏移属性来改变其位置。

display 属性

- 设定display属性值是在CSS中实现页面布局的一种主要方法，此属性允许更改默认显示方式

值	描述
none	此元素不会被显示。
block	此元素将显示为块级元素，此元素前后会带有换行符。
inline	默认。此元素会被显示为内联元素，元素前后没有换行符。
inline-block	行内块元素。（CSS2.1 新增的值）
list-item	此元素会作为列表显示。
run-in	此元素会根据上下文作为块级元素或内联元素显示。
compact	CSS 中有值 compact，不过由于缺乏广泛支持，已经从 CSS2.1 中删除。
marker	CSS 中有值 marker，不过由于缺乏广泛支持，已经从 CSS2.1 中删除。
table	此元素会作为块级表格来显示（类似 <table>），表格前后带有换行符。
inline-table	此元素会作为内联表格来显示（类似 <table>），表格前后没有换行符。
table-row-group	此元素会作为一个或多个行的分组来显示（类似 <tbody>）。
table-header-group	此元素会作为一个或多个行的分组来显示（类似 <thead>）。
table-footer-group	此元素会作为一个或多个行的分组来显示（类似 <tfoot>）。
table-row	此元素会作为一个表格行显示（类似 <tr>）。
table-column-group	此元素会作为一个或多个列的分组来显示（类似 <colgroup>）。
table-column	此元素会作为一个单元格列显示（类似 <col>）
table-cell	此元素会作为一个表格单元格显示（类似 <td> 和 <th>）
table-caption	此元素会作为一个表格标题显示（类似 <caption>）
inherit	规定应该从父元素继承 display 属性的值。

display属性

- **display:none** 用来隐藏元素
 - **SEO技巧**: 搜索引擎会忽视display:none的内容
 - **display:none vs visibility:hidden**

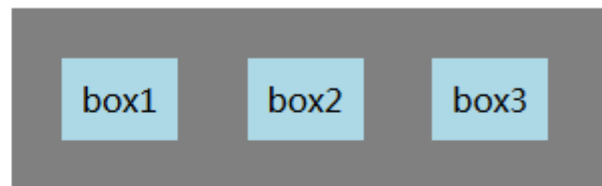
在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father {
7       width:280px; padding: 10px;
8       background-color:grey;    }
9     #father div {
10      padding:10px; margin:15px;
11      display:inline-block;
12      background-color:lightblue;
13    }
14
15  </style>
16 </head>
17 <body>
18   <div id="father">
19     <div id="son1">box1</div>
20     <div id="son2">box2</div>
21     <div id="son3">box3</div>
22   </div>
23 </body>
24 </html>
```

The Foundation of Web Development

显示效果



#father #son2{display:none;}



#father #son2{visibility:hidden;}

display 属性

• 示例：利用组合器和伪类实现下拉菜单【回顾】

- 选择所有嵌套在 `div.menu-bar` 中的 `<ul>` 元素，并将它们的显示设置为隐藏，也就是不可见。
- 当鼠标悬停在 `div.menu-bar` 中的任意 `<li>` 元素上时，将标识设置为红色。
- 当鼠标悬停在 `div.menu-bar` 中的 `<li>` 元素上时，这个 `li` 的直接子元素 `ul` 会被选中，并应用定义的样式规则。

```
<div class="menu-bar">
  <ul>
    <li>
      <a href="example.html">Menu</a>
      <ul>
        <li>
          <a href="example.html">Link</a>
        </li>
        <li>
          <a class="menu-nav" href="example.html">Submenu</a>
          <ul>
            <li>
              <a class="menu-nav" href="example.html">Submenu</a>
              <ul>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
                <li><a href="example.html">Link</a></li>
              </ul>
            </li>
            <li><a href="example.html">Link</a></li>
          </ul>
        </li>
      </ul>
    </li>
  </ul>
</div>
```

```
div.menu-bar ul ul {
  display: none;
}
div.menu-bar li:hover {
  color: red;
}
div.menu-bar li:hover > ul {
  display: block;
  color: green;
}
```



尝试：把 `>` (子元素选择器) 换成空格 (后代选择器) 会怎么样？

A：下拉菜单会一次全部显示

这个例子表现了后代和子元素的区别

display 属性

- 利用列表和链接做导航菜单效果【回顾】

Home

Pizza

Music

Wombats

Finland

```
<ul>
  <li><a href="#">Home</a></li>
  <li><a href="#">Pizza</a></li>
  <li><a href="#">Music</a></li>
  <li><a href="#">Wombats</a></li>
  <li><a href="#">Finland</a></li>
</ul>
```

```
body,html {
  margin: 0;
  font-family: sans-serif;
}
```

```
ul {
  padding: 0;
  width: 100%;
}
```

```
li {
  display: inline;
}
```

```
a {
  outline: none;
  text-decoration: none;
  display: inline-block;
  width: 19.5%;
  margin-right: 0.1%;
  text-align: center;
  line-height: 3;
  color: black;
}
```

```
a:link, a:visited, a:focus {
  background: yellow;
}

a:hover {
  background: orange;
}

a:active {
  background: red;
  color: white;
}
```

display 属性

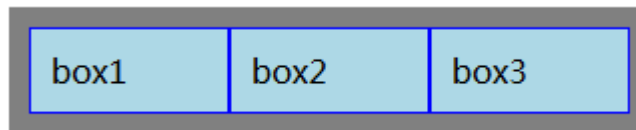
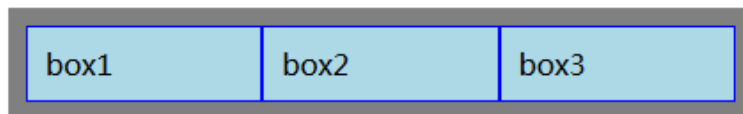
- 自动平均划分元素（把一个块级元素的显示方式改成了表格单元格模式）
 - 父元素 {display: table;}
 - 子元素 {display: table-cell;}

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father {
7       width:400px; padding: 10px;
8       background-color:grey;
9       display:table;}
10    #father div {
11      padding:10px; margin:15px;
12      display:table-cell;
13      background-color:lightblue;
14      border: 1px solid blue;
15    }
16  </style>
17 </head>
18 <body>
19   <div id="father">
20     <div id="son1">box1</div>
21     <div id="son2">box2</div>
22     <div id="son3">box3</div>
23   </div>
24 </body>
25 </html>
```

显示效果



#father { width=300px; ... }

div元素会自动平均划分宽度，
不需要指定宽度

display属性

- 去除inline-block元素间距

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6   <style>
7     ul,li,a{
8       padding:0; margin:0;
9       list-style: none;
10      text-decoration: none;}
11   li{ display: inline-block;
12      border: thin solid black;
13      background-color:lightgrey;
14      font-size:32px; }
15   </style>
16 </head>
17 <body>
18   <ul>
19     <li><a href="">1</a></li>
20     <li><a href="">2</a></li>
21     <li><a href="">3</a></li>
22     <li><a href="">4</a></li>
23     <li><a href="">5</a></li>
24   </ul>
25 </body>
26 </html>
```

显示效果

1 2 3 4 5



12345

ul {font-size:0;}

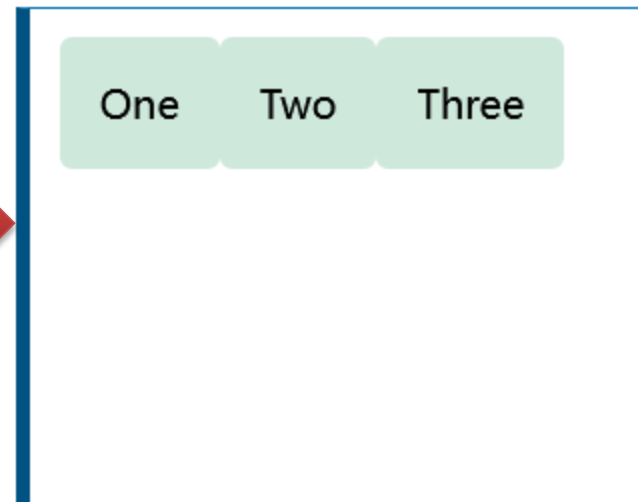
通过设置父元素font-size:0;来去除inline-block元素的间距

利用display实现Flexbox布局

- Flexbox 是CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 它被专门设计出来用于创建横向或是纵向的一维页面布局
- 要使用flexbox, 需要在想要进行flex布局元素的父元素 (称为 flex 容器, flex container) 上应用display: flex, 所有直接子元素都将会按照flex进行布局

```
.wrapper {  
  display: flex;  
}
```

```
<div class="wrapper">  
  <div class="box1">One</div>  
  <div class="box2">Two</div>  
  <div class="box3">Three</div>  
</div>
```



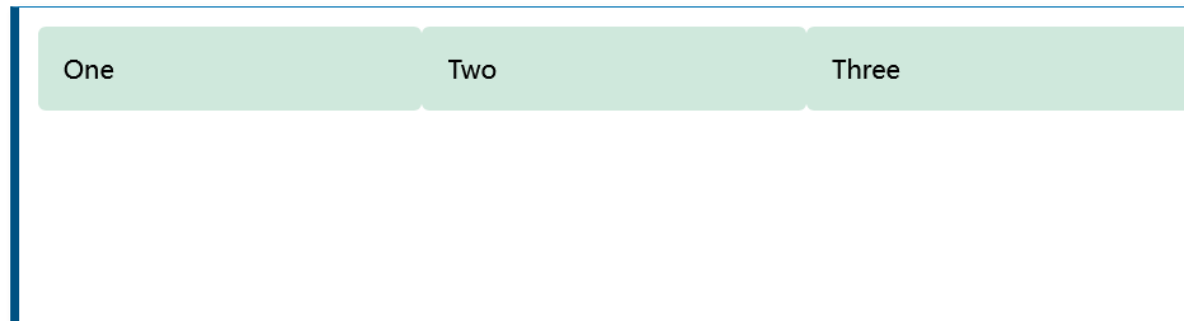
利用display实现Flexbox布局

- 除了可以应用到**flex 容器**的属性以外，还有很多属性可以应用到**flex 项**(flex items)上面，这些属性可以改变flex项在flex布局中占用宽/高的方式，允许它们通过伸缩来适应可用空间

```
.wrapper {  
  display: flex;  
}
```

```
.wrapper > div {  
  flex: 1;  
}
```

每个子元素平分父容器的剩余空间。



```
<div class="wrapper">  
  <div class="box1">One</div>  
  <div class="box2">Two</div>  
  <div class="box3">Three</div>  
</div>
```

利用display实现Grid布局

- Flexbox用于设计横向或纵向的布局，而Grid布局则被设计用于**同时在两个维度**上把元素按行和列排列整齐
- 可以通过指定display:grid来实现grid布局，还可以使用 grid-template-rows 和 grid-template-columns 两个属性定义行和列

```
.wrapper {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 100px 100px;  
  grid-gap: 10px;  
}
```

```
<div class="wrapper">  
  <div class="box1">One</div>  
  <div class="box2">Two</div>  
  <div class="box3">Three</div>  
  <div class="box4">Four</div>  
  <div class="box5">Five</div>  
  <div class="box6">Six</div>  
</div>
```

利用display实现Grid布局

- Flexbox用于设计横向或纵向的布局，而Grid布局则被设计用于**同时在两个维度**上把元素按行和列排列整齐
- 可以通过指定display:grid来实现grid布局，还可以使用 grid-template-rows 和 grid-template-columns 两个属性定义行和列

One

Two

Three

Four

Five

Six

学习Javascript



回顾Web标准

- Web标准并不是某一个标准，而是一系列标准的集合，主要包括结构（Structure）、呈现（Presentation）和行为（Behavior）三个方面
 - 结构标准：HTML、XML、XHTML
 - 呈现标准：CSS
 - 行为标准：DOM、ECMAScript
 - 行为是指网页模型的定义及交互的编写，主要包括DOM和ECMAScript。
 - DOM（英文Document Object Model的缩写）是文档对象模型。W3C 文档对象模型（DOM）是中立于平台和语言的接口，它允许程序和脚本动态地访问和更新文档的内容、结构和样式
 - ECMAScript是ECMA国际以JavaScript为基础制定的标准脚本语言。JavaScript是一种基于对象和事件驱动的客户端脚本语言，广泛用于Web开发，用来给HTML网页添加动态功能，如响应用户操作

从一个实例开始



问题： 在一个网页中，HTML、CSS和Javascript是怎么发挥作用的呢？

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
5  <title>JS彩色方块倒计时代码 - A5源码</title>
6  <style>...
21 </style>
22 <script type="text/javascript" src="js/timecountdown.js"></script>
23 <script type="text/javascript">...
57 </script> -->
58 </head>
59
60 <body>
61 <div>...
89 </div>
90
91 <div>...
115 </div>
116
117 <div>
118   <i>60秒倒计时</i>
119   <div>
120     <span class="pk">
121       <p id="s3"></p>
122       <p>秒</p>
123     </span>
124     <span class="mk">
125       <p id="m3"></p>
126       <p>毫秒</p>
127     </span>
128   </div>
129 </div>
130
131 </body>
132 </html>

```

样式

行为

内容

从一个实例开始

```
6 <style>
7   *{padding: 0; margin: 0; font-family: "Lato", sans-serif}
8   i{font-size: 14px}
9   body{background: ■ #0d0d0d; color: □ #ffffff}
10  div{font-size: 20px; text-align: center; padding-top: 15px;}
11  div span{display: inline-block; width: 50px; height: 50px; padding: 10px; border-radius: 5px}
12  p{font-size: 18px}
13  span.red{background: ■ #ff0000}
14  span.yg{background: ■ yellowgreen}
15  span.or{background: ■ orange}
16  span.pi{background: ■ palevioletred}
17  span.pk{background: ■ #00a0e9}
18  span.mk{background: ■ #1FADC5}
19  span p:first-child{font-size: 25px}
20  span p:last-child{font-size: 14px}
21 </style>
22 <script type="text/javascript" src="js/timecountdown.js"></script>
23 ⊕ <script type="text/javascript">...
57 </script>
```

问题：请仔细观察，这些CSS规则有什么作用？
如果禁用CSS，网页效果会是什么样的？

从一个实例开始

距2028年11月11日还有

9

年

355

天

12

时

49

分

44

秒

776

毫秒

问题：禁用CSS的网页效果是这个样子的，和之前有什么区别？

从一个实例开始

```
<script type="text/javascript" src="js/timecountdown.js"></script>
<script type="text/javascript">
/**
 * time.day 和 time.dayZero 区别
 * time.day 如果是1天 返回 1
 * time.dayZero 如果是1天 则返回 01
 * 除了day拥有Zero外 hour,minute,second,msec都有Zero 即小于10的, 都会在前面补0
 */
window.onload=function(){
    xcsoft.countdown('2028-11-11',function (time) {
        //time.days=总天数
        document.getElementById("y1").innerHTML=time.year
        document.getElementById("d1").innerHTML=time.day
        document.getElementById("h1").innerHTML=time.hourZero
        document.getElementById("i1").innerHTML=time.minuteZero
        document.getElementById("s1").innerHTML=time.secondZero
        document.getElementById("m1").innerHTML=time.msecZero
    },function (time) {
        //倒计时结束后的操作
    })
    xcsoft.countup('2018-11-11',function (time) {
        document.getElementById("y2").innerHTML=time.year
        document.getElementById("d2").innerHTML=time.day
        document.getElementById("h2").innerHTML=time.hourZero
        document.getElementById("i2").innerHTML=time.minuteZero
        document.getElementById("s2").innerHTML=time.secondZero
    })
    var nes=parseInt(new Date().getTime()/1000)+60;
    xcsoft.countdown(parseInt(nes)+'.3',function (time) {
        document.getElementById("s3").innerHTML=time.secondZero
        document.getElementById("m3").innerHTML=time.msecZero
    })
}
</script>
```

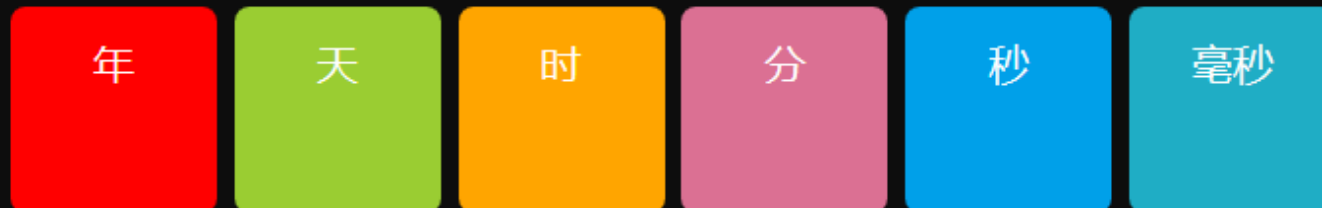
timecountdown.js

```
xcsoft.countdown = function(a, b, c) {
    var d = new Object();
    var e = 0;
    if (typeof a == "object") {
        var f = parseInt(new Date().getTime() / 1000);
        var g = a.startTime ? parseInt(a.startTime) : 0;
        g = g == 0 ? f : g;
        var h = a.endTime;
        var x = g - f;
        d.decimal = parseInt(a.msec ? a.msec : 0)
    } else {
        var f = 0;
        var x = 0;
        d.decimal = 0;
        var h = a;
        var i = h.toString();
        if (i.indexOf('.') > 0) {
            d.decimal = i.split('.')[1];
            if (d.decimal > 3) {
                d.decimal = 3
            }
        }
    }
    d.time = h;
    d.finish = false;
    if (isNaN(h)) {
        var j = h.substring(0, 19);
        j = j.replace(/-/g, '/');
        d.time = new Date(j).getTime() / 1000
    }
}
```

问题：请仔细观察这些js代码，有什么发现？猜测一下，它们有什么作用？禁用javascript，网页效果会是什么样的？

从一个实例开始

距2028年11月11日还有



从2018年11月11日至今已经过去



60秒倒计时



JavaScript 导论

- JavaScript 是一种轻量级的脚本语言 (script language)，也是一种嵌入式 (embedded) 语言
- 所谓“脚本语言” (script language)，指的是它不具备开发操作系统的能力，而是只用来编写控制其他大型应用程序(比如浏览器)的“脚本”
- JavaScript 本身提供的核心语法不算很多，只能用来做一些数学和逻辑运算，JavaScript 本身不提供任何 I/O 相关的 API，要靠宿主环境 (host) 提供，JavaScript 只合适嵌入更大型的应用程序环境，去调用宿主环境提供的底层 API
 - 已经嵌入 JavaScript 的宿主环境有多种，最常见的就是浏览器，另外还有服务器环境，也就是 Node 项目

JavaScript 导论

- JavaScript 的核心语法部分相当精简，只包括两个部分：基本的语法构造（比如操作符、控制结构、语句）和标准库（就是一系列具有各种功能的对象比如Array、Date、Math等）

JavaScript 导论

- 除此之外，各种宿主环境提供额外的 API（即只能在该环境使用的接口），以便 JavaScript 调用
 - 如果宿主环境是浏览器，提供的额外 API 可以分成三大类，①浏览器控制类：操作浏览器；②DOM 类：操作网页的各种元素；③Web 类：实现互联网的各种功能
 - 如果宿主环境是服务器，则会提供各种操作系统的 API，比如文件操作 API、网络通信 API 等等。这些都可以在 Node 环境中找到

学习JavaScript的优势

- 操控浏览器的能力
- 广泛的使用领域
- 易学性
- 强大的性能
- 开放性
- 社区支持和就业机会
 - 前端开发
 - 后端开发
 - 全栈开发
 - 移动应用开发
 - 游戏开发
 - 数据可视化

操控浏览器的能力

- JavaScript 的发明目的，就是作为浏览器的内置脚本语言，为网页开发者提供操控浏览器的能力
 - 它是目前通用的浏览器脚本语言，所有浏览器都支持。它可以让网页呈现各种特殊效果，为用户提供良好的互动体验
 - 目前，全世界几乎所有网页都使用 JavaScript。如果不用，网站的易用性和使用效率将大打折扣，无法成为操作便利、对用户友好的网站
 - 对于一个互联网开发者来说，如果你想提供漂亮的网页、令用户满意的上网体验、各种基于浏览器的便捷功能、前后端之间紧密高效的联系，JavaScript 是必不可少的工具

广泛的使用领域

- 近年来，JavaScript 的使用范围慢慢超越了浏览器，正在向通用的系统语言发展

— 浏览器的平台化

- 随着 HTML5 的出现，浏览器本身的功能越来越强，不再仅仅能浏览网页，而是**越来越像一个平台**，JavaScript 因此得以调用许多系统功能，比如操作本地文件、操作图片、调用摄像头和麦克风等等。越来越多的应用程序选择在浏览器中运行，而不是传统的桌面应用程序。这种趋势被称为“Web化”，它推动了浏览器成为应用程序平台的发展。

— Node

- Node 项目使得 JavaScript 可以用于开发服务器端的大型项目，网站的前后端都用 JavaScript 开发已经成为了现实。甚至可以应用到嵌入式平台（Raspberry Pi）之上（它通常以单板计算机的形式出现，尺寸较小，价格低廉，但性能仍然强大。Raspberry Pi 主要采用 Linux 操作系统，并且可以运行多种应用程序，包括办公软件、编程工具、媒体中心等。）

— 数据库操作

- JavaScript 甚至也可以用来操作数据库。NoSQL 数据库这个概念，本身就是在 JSON（JavaScript Object Notation）格式的基础上诞生的，大部分 NoSQL 数据库允许 JavaScript 直接操作。基于 SQL 语言的开源数据库 PostgreSQL 支持 JavaScript 作为操作语言，可以部分取代 SQL 查询语言

广泛的使用领域

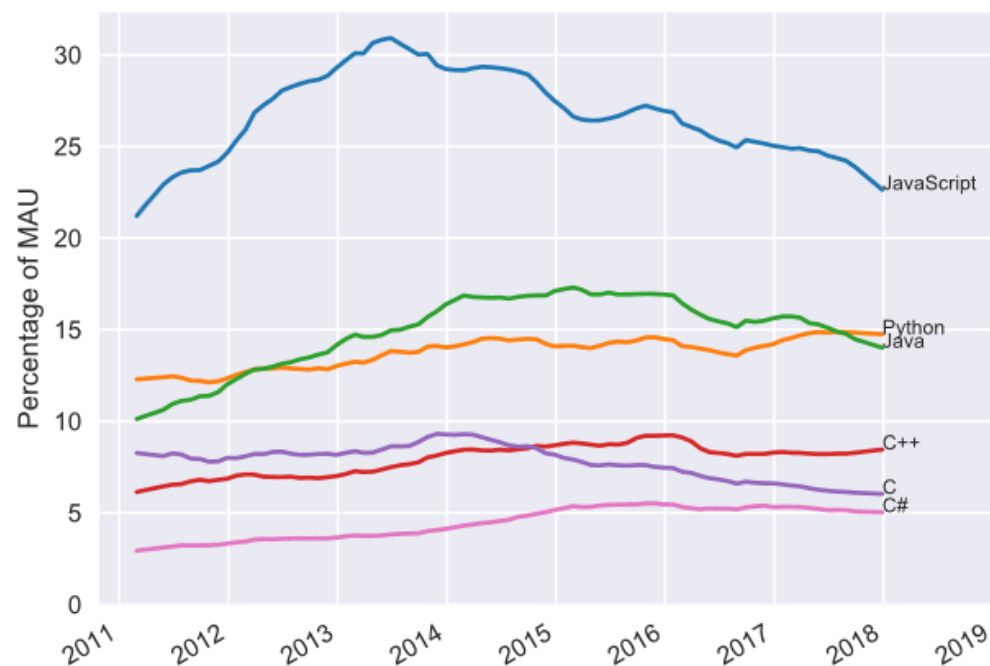
- 近年来，JavaScript 的使用范围慢慢超越了浏览器，正在向通用的系统语言发展
 - 移动平台开发
 - JavaScript 也正在成为手机应用的开发语言，许多人正在努力，让 JavaScript 成为覆盖安卓、iOS各个平台的通用开发语言
 - 内嵌脚本语言
 - 越来越多的应用程序，将 JavaScript 作为内嵌的脚本语言，比如 Adobe 公司的著名 PDF 阅读器 Acrobat、Linux 桌面环境 GNOME 3
 - 跨平台的桌面应用程序
 - Chromium OS、Windows 8 等操作系统直接支持 JavaScript 编写应用程序。Mozilla 的 Open Web Apps 项目、Google 的 Chrome App 项目、Github 的 Electron 项目、以及 TideSDK 项目，都可以用来编写运行于 Windows、Mac OS 和 Android 等多个桌面平台的程序，不依赖浏览器

广泛的使用领域

根据历史数据，从2014-2024年的排名来看：JavaScript长期占据首位。

Ranking Programming Languages by GitHub Users

Rank	Language	Monthly Active Users	Trend
1	JavaScript	22.63%	
2	Python	14.75%	
3	Java	14.01%	
4	C++	8.45%	
5	C	6.03%	
6	PHP	5.85%	
7	C#	5.03%	
8	Shell	4.85%	
9	Go	4.10%	
10	TypeScript	3.89%	



广泛的使用领域

- **JavaScript 最终将能让你只用一种语言，就开发出适应不同平台（包括桌面端、服务器端、手机端）的程序**

著名程序员 Jeff Atwood 甚至提出了一条 “Atwood 定律”：

“所有可以用 JavaScript 编写的程序，最终都会出现 JavaScript 的版本。”(Any application that can be written in JavaScript will eventually be written in JavaScript.)

易学性

- 学习环境无处不在

- 只要有浏览器，就能运行 JavaScript 程序；只要有文本编辑器，就能编写 JavaScript 程序。这意味着，几乎所有电脑都原生提供 JavaScript 学习环境，不用另行安装复杂的 IDE 和编译器

- 简单性及主流语言相似性

- JavaScript 的语法相比其他脚本语言（如 Python 或 Ruby）更简单一些，本身语法特性并不特别多
 - JavaScript 的语法类似 C/C++ 和 Java，入门容易

易学性

- 必须说明的是，虽然核心语法不难，但是 JavaScript 的复杂性体现在另外两个方面
 - 首先，它涉及大量的外部 API。JavaScript 要发挥作用，必须与其他组件配合，这些外部组件五花八门，数量极其庞大，几乎涉及网络应用的各个方面，掌握它们绝非易事
 - 其次，JavaScript 语言有一些设计缺陷。某些地方相当不合理，另一些地方则会出现怪异的运行结果。学习 JavaScript，很大一部分时间是用来搞清楚哪些地方有陷阱
- 尽管如此，目前看来，JavaScript 的地位还是无法动摇。加之，语言标准的快速进化，使得 JavaScript 功能日益增强，而语法缺陷和怪异之处得到了弥补。所以，JavaScript 还是值得学习，况且它的入门真的不难



当心坑洞

强大的性能

- 灵活的语法，表达力强。
 - JavaScript 既支持类似 C 语言清晰的**过程式编程**，也支持灵活的**函数式编程**，可以用来写并发处理。这些语法特性已经被证明非常强大，可以用于许多场合，尤其适用异步编程

进一步阅读： 函数式编程是一种编程范式（programming paradigm），也就是如何编写程序的方法论。它属于结构化编程的一种，主要思想是**把运算过程尽量写成一系列嵌套的函数调用**。函数式编程具有五个鲜明的特点：

1. 函数是“第一等公民”、2. 只用“表达式”，不用“语句”、3. 没有“副作用”、4. 不修改状态、5. 引用透明

https://www.ruanyifeng.com/blog/2012/04/functional_programming.html

```
var result = subtract(multiply(add(1,2), 3), 4);
```

强大的性能

- 支持编译运行

- JavaScript 虽然是一种解释型语言，但是在现代浏览器中，JavaScript 都是**编译后运行**。程序会被高度优化，运行效率接近二进制程序
- WebAssembly 是一种主流浏览器都已经支持的新的字节码格式，它是 JavaScript 引擎的**中间码格式**，全部都是二进制代码。由于跳过了编译步骤，可以达到**接近原生二进制代码的运行速度**

进一步阅读：编译语言的程序在运行时效率比脚本语言高，因为代码已经被转化为机器语言，可以直接被计算机硬件执行。而脚本语言的程序每次执行都需要通过解释器进行解释，效率相对低一些。在编译语言中，可以将代码静态检查，编译错误可以在编译期间发现并修改。而脚本语言在运行时才会发现错误。

强大的性能

- 事件驱动和非阻塞式设计

- JavaScript 程序可以采用事件驱动 (event-driven) 和非阻塞式 (non-blocking) 设计，在服务器端适合高并发环境，普通硬件就可以承受很大的访问量

1.事件驱动：事件驱动是指程序的执行流程是由事件触发来驱动的，而不是线性地按照代码顺序执行。在JavaScript中，各种交互事件（比如点击、鼠标移动、键盘输入等）以及其他异步操作（比如定时器、HTTP请求等）都可以作为事件，当这些事件发生时，对应的回调函数将被执行。这种方式使得JavaScript程序可以响应用户操作和外部事件，而无需等待某个任务的完成才能执行下一个任务。

2.非阻塞式IO：JavaScript程序的非阻塞式设计使得在执行IO操作（如文件读写、网络请求等）时，不会因为等待IO操作完成而阻塞其他任务的执行，而是通过回调函数或Promise等方式在IO操作完成后再进行处理。这样可以更有效地利用服务器的资源，避免因IO操作阻塞而导致的性能问题。

3.事件循环机制：JavaScript通过事件循环机制实现异步任务的调度和执行，这样可以在单线程的情况下处理大量并发的IO操作，从而减少了线程切换的开销，提高了服务器对并发请求的处理能力。

开放性

- JavaScript 是一种开放的语言，标准 ECMA-262 是 ISO 国际标准，语言标准由 TC39 委员会负责制定，该委员会的运作是透明的，所有讨论都是开放的，会议记录都会对外公布



Standard ECMA-262
5.1 Edition / June 2011

ECMAScript® Language Specification

开放性

- ECMA-262标准的主要实现都是开放的，不同公司的 JavaScript 运行环境质量很高，兼容性好
 - 首个 JavaScript 引擎由网景公司的 Brendan Eich 创建，引擎代号 **SpiderMonkey**，由 C/C++ 语言开发
 - Rhino 引擎，主要由网景公司的 Norris Boyd 创建，由 Java 语言开发
 - Google 的 **V8**，在 Google Chrome 浏览器和较新的 Opera 浏览器中使用。同时也是 Node.js 使用的引擎
 - JavaScriptCore (SquirrelFish/Nitro)，被用在了一些 WebKit 浏览器如 Apple Safari
 - Carakan，用在旧版本 Opera 中
 - The Chakra 引擎，用在 Internet Explorer 中

社区支持和就业机会

- 全世界程序员都在使用 JavaScript，它有着极大的社区、广泛的文献和图书、丰富的代码资源。绝大部分你需要用到的功能，都有多个开源函数库可供选用
- 作为项目负责人，你不难招聘到数量众多的 JavaScript 程序员；作为开发者，你也不难找到一份 JavaScript 的工作

学习JavaScript的优势

- 操控浏览器的能力、广泛的使用领域
- 易学性、强大的性能、开放性
- 社区支持和就业机会

拥有
强大的
生命力

JavaScript和Java
什么关系?

Javascript和
ECMAScript有什么关系?

JavaScript和浏览
器什么关系?

Mozilla和Netscape
有什么关系?



JavaScript 语言的历史

- JavaScript是Mozilla 项目联合创始人， Mozilla 基金会和 Mozilla 公司的 Brendan Eich用10天设计出来的
 - 基本语法：借鉴C语言和Java语言
 - 数据结构：借鉴Java语言，包括将值分成原始值和对象两大类
 - 函数的用法：借鉴Scheme语言和Awk语言，将函数当作第一等公民，并引入闭包
 - 原型继承模型：借鉴Self语言（Smalltalk的一种变种）。
 - 正则表达式：借鉴Perl语言
 - 字符串和数组处理：借鉴Python语言



作为设计者，他一点也不喜欢自己的这个作品：

"与其说我爱JavaScript，不如说我恨它。它是C语言和Self语言一夜情的产物。十八世纪英国文学家约翰逊博士说得好：'它的优秀之处并非原创，它的原创之处并不优秀。'(the part that is good is not original, and the part that is original is not good.)"

JavaScript 语言的历史

- 回顾JavaScript的历史，要从浏览器历史讲起
 - 1990年底，欧洲核能研究组织（CERN）科学家 Tim Berners-Lee发明了WWW
 - 1992年底，美国国家超级电脑应用中心（NCSA）开始开发Mosaic浏览器，第一款图形界面浏览器
 - 1994年10月，NCSA 的主要程序员 Marc Andreessen 联合风投家 Jim Clark，成立了 Mosaic 通信公司，不久后改名为 Netscape，在 Mosaic 的基础上，开发面向普通用户的新一代的浏览器 Netscape Navigator，1994年12月，Navigator 发布了1.0版

JavaScript 语言的历史

- Netscape 公司很快发现，Navigator 浏览器需要一种可以嵌入网页的脚本语言，用来控制浏览器行为。当时，网速很慢而且上网费很贵，有些操作不宜在服务器端完成。Netscape 公司决定与 Sun 公司合作，浏览器支持嵌入 Java 小程序（后来称为 Java applet）
- 浏览器脚本语言是否就选用 Java，则存在争论。后来，还是决定不使用 Java，因为网页小程序不需要 Java 这么“重”的语法。但是，同时也决定脚本语言的语法要接近 Java，并且可以支持 Java 程序。这些设想直接排除了使用现存语言，比如 Perl、Python 和 TCL（Tool Command Language）

JavaScript 语言的历史

- 1995年，Netscape 公司雇佣 34岁的Brendan Eich 开发这种网页脚本语言，Brendan Eich的主要方向和兴趣是函数式编程
- 网景公司做出决策，未来的网页脚本语言必须"看上去与Java足够相似"，但是比Java简单，Brendan Eich被指定为这种"简化版Java语言"的设计师。但是，他对Java一点兴趣也没有。为了应付公司安排的任务，他只用10天时间就把Javascript设计出来了
- Javascript语言实际上是两种语言风格的混合产物---(简化的)函数式编程+(简化的)面向对象编程。 这是由Brendan Eich(函数式编程)与网景公司(面向对象编程)共同决定的。

JavaScript 语言的历史

- Netscape 公司的这种浏览器脚本语言，最初名字叫做 Mocha，1995年9月改为 LiveScript。12月，Netscape 公司与 Sun 公司（Java 语言的发明者和所有者）达成协议，后者允许将这种语言叫做 JavaScript
- 1995年12月4日，Netscape 公司与 Sun 公司联合发布 JavaScript 语言，对外宣传 JavaScript 是 Java 的补充，属于轻量级的 Java，专门用来操作网页
- 1996年3月，Navigator 2.0 浏览器正式内置 JavaScript 脚本语言

JavaScript 与 Java 的关系

- JavaScript 和 Java 是两种不一样的语言，JavaScript 的基本语法和对象体系，是模仿 Java 而设计的
- JavaScript 语言的函数是一种独立的数据类型，以及采用基于原型对象（prototype）的继承链。这是它与 Java 语法最大的两点区别
 - 在JavaScript中，函数是“第一等公民”（first class），函数与其他数据类型一样，处于平等地位，可以赋值给其他变量，也可以作为参数，传入另一个函数，或者作为别的函数的返回值
 - JavaScript 常被描述为一种基于原型的语言（prototype-based language）——每个对象拥有一个原型对象，对象以其原型为模板、从原型继承方法和属性。原型对象也可能拥有原型，并从中继承方法和属性，一层一层、以此类推。这种关系常被称为原型链（prototype chain），它解释了为何一个对象会拥有定义在其他对象中的属性和方法。
- Java 语言需要编译，而JavaScript 语言则是运行时由解释器直接执行。

JavaScript 与 ECMAScript 的关系

- 1996年8月，微软模仿 JavaScript 开发了一种相近的语言，取名为 JScript（JavaScript 是 Netscape 的注册商标，微软不能用），首先内置于 IE 3.0。Netscape 公司面临丧失浏览器脚本语言的主导权的局面
- 1996年11月，Netscape 公司决定将 JavaScript 提交给国际标准化组织 ECMA（欧洲计算机制造商协会，European Computer Manufacturers Association），希望 JavaScript 能够成为国际标准，以此抵抗微软

JavaScript 与 ECMAScript 的关系

- 1997年7月，ECMA 组织发布262号标准文件（ECMA-262）的第一版，规定了浏览器脚本语言的标准，并将这种语言称为 ECMAScript。这个版本就是 ECMAScript 1.0 版
- 之所以不叫 JavaScript，一方面是由于商标的关系，Java 是 Sun 公司的商标，根据一份授权协议，只有 Netscape 公司可以合法地使用 JavaScript 这个名字，且 JavaScript 已经被 Netscape 公司注册为商标，另一方面也是想体现这门语言的制定者是 ECMA，不是 Netscape，这样有利于保证这门语言的开放性和中立性
- ECMAScript 和 JavaScript 的关系是，前者是后者的规格，后者是前者的一种实现

Mozilla和Netscape的关系

- Mozilla是Netscape Navigator项目的内部开发代号
- Mozilla.org于1998年America Online（美国在线，AOL）收购Netscape后成立，旨在进行Netscape一直推广的开源项目
- 2003年，Netscape公司被解散，部分员工组成了Mozilla基金会，并开发出Firefox

JavaScript 版本

Version	Release Date	Major Changes
1	1997年7月	Initial release.
2	1998年6月	主要是进行编辑更改，以使规范与ISO/IEC 16262国际标准（ECMAScript）保持一致
3	1999年12月	增加了对正则表达式的支持，改进了字符串处理，添加了新的控制语句、异常处理和各种数字输出格式等功能
4	--	由于政治分歧和语言复杂性的技术困难，该项目被放弃。其中一些开发的功能被纳入了第5版，而其他一些则推迟到未来的版本中。
5	2009年12月	添加了严格模式，澄清了模糊之处，并对实现的真实世界用法进行了调整。增加了getter和setter，支持JSON和对对象属性的反射。
5.1	2011年6月	5.1发布以配合国际标准ISO/IEC 16262:2011的第三版。所有主要浏览器都支持 ECMAScript 5.1版的全部功能
6	2015年6月	为构建复杂应用程序引入了重要的新语法，包括长期期待的对类（class）和模块（module）的直接语言支持，以及众多其他语言增强功能。该版本更名为 ECMAScript 2015。

实操作业-设计个性化校园卡

我的校园卡（可以任意改变）

姓名 学号



要求：要素、形状、布局保留，颜色、背景图片可自由发挥各位的想象力，个性化、美观、大方...

实操作业-设计个性化校园卡

- 作业要求（独立完成，汇总后提交）：
 - 提交压缩文件，姓名_学号.rar\zip\7z\tar.gz：
 - 符合要求的网页及网页源码、文件目录、HTML、CSS（引用方式自定）、图片资源等内容
 - 提交时间：
 - 用时二周，11月24日前提交到教学云平台
 - 考察内容
 - 网页的文件组织
 - HTML的基本结构及元素
 - CSS的引用方式、双栏布局、盒模型、定位、选择器、样式、如何应对浏览器放缩？跨浏览器测试

网安小知识--朝鲜方向Lazarus

•攻击者以安全研究人员的身份，至少从2020年4月份开始潜伏在 Twitter、LinkedIn等社交媒体

•攻击者与目标安全研究人员通过社交媒体建立联系后，开始进行社会工程学攻击，其会邀请目标进行漏洞研究方面的合作，向目标发送相关 Visual Studio 项目，而该项目一旦编译则会触发执行恶意模块，最终被攻击者控制，具有很强的隐蔽性。

Exploit Research Blog

HOME

BLOG

Search in blog

DOS2RCE: A New Technique to Exploit V8
NULL Pointer Dereference Bug

Posted on: Tue, Oct 13 2020 by Vanya Willy

Issue **0x00286: Security: A null pointer dereference has been discovered in V8 compiler which affects the latest version. This vulnerability allows an attacker to bypass a security mitigation.**
Reported by [secur](#). [@hacker.gv.uk on the Sep. 12, 2019, 3:28 AM PDT](#)

NCSC Contact Information
The vulnerability disclosure mailbox is ["security@ncsc.gov.uk"](#). Please contact us for our PGP key.

Crediting NCSC
NCSC would appreciate appropriate credit as The UK's National Cyber Security Centre (NCSC) in any advisories which you may

Archive (46)

2020 (3)

November (1)

October (2)

Introduction

Last year, The UK's National Cyber Security Centre (NCSC) reported a bug existed in the V8 compiler and Google patched it shortly. The bug ID is 0x00286 and you can find the details here. According to their report, it is a NULL pointer dereference DoS bug which is not exploitable and it can be triggered via WASM code. During the deep analysis, we found that there is another way to trigger the bug and it is exploitable by leveraging V8 JIT compiler process.

In this blog, we describe the details to exploit this vulnerability and demonstrate the remote code execution.

Root Cause

For the code optimization, V8 JIT compiler uses the Node graphs and generates optimized native code by reducing them through several phase of optimization pipelines. This Node graphs are also used in WASM compiler to compile the WASM code to native code.

The Nodes are linked each other in the graph using "Use" structure as following.

The screenshot shows a WhatsApp chat interface. At the top, the status bar shows the time 20:14, battery level at 46%, and signal strength. The chat header shows the contact name 'emp pilot' with a search icon and a status 'last seen yesterday'. The chat is dated 'January 10'.

The conversation begins with a message from 'Zhang Guo' (blue bubble): 'So I'm looking for someone to have experience for windows kernel.' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Then, a message from 'emp pilot' (green bubble) appears: 'Hi' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Next, a message from 'Zhang Guo' (blue bubble) says: 'I'm going to exploit my windows daydy.' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Then, a message from 'emp pilot' (green bubble) says: 'Can you help me?' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Next, a message from 'Zhang Guo' (blue bubble) says: 'I really should not be able to help you, because I am not great at writing code.' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Then, a message from 'emp pilot' (green bubble) says: 'thanks for your kind answers.' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Next, a message from 'Zhang Guo' (blue bubble) says: 'See u later.' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

Then, a message from 'emp pilot' (green bubble) says: 'Bye!' (20:17 16.30). This is followed by a read receipt (20:17 16.30) and a typing indicator (20:17 16.30).

At the bottom of the chat, there is a navigation bar with icons for 'Message', 'Voice', 'Video', and 'Share'. The 'Message' icon is highlighted.

James Willy @jamesw0x0 · Jan 14
 I think this is not a fake video

Kevin Lopez @mvp43r · Jan 14
 I have just exploited CVE-2021-1647 Windows defender Oday vulnerability
 #Oday #exploit #microsoft #Windows Security
[youtube.com/watch?v=FlaYEn...](https://www.youtube.com/watch?v=FlaYEn...)

Pinot Tweet

Zhang Guo @z0x55g · Nov 5, 2020
 My WebAssembly Engine Remote Code Execution Exploit Demo on Chrome 86.0.4240.75
[#Exploit #Webassembly #PwB #Security](#)

WebAssembly Engine Remote Code Execution Exploit Demo
[@j0y0ur.com](#)

2020/12/17	15:13:28	EGCzcZYW8OC77yZtAZIL	中国	香港	tencent.com
2020/12/17	15:13:36	5i7eF9BH8a980Aqn6	中国	浙江	杭州 电信
2020/12/19	7:41:48	5i7eF9BH8a980Aqn6	中国	香港	
2020/12/21	0:33:40	5i7eF9BH8a980Aqn6	中国	香港	
2020/12/21	0:34:12	5i7eF9BH8a980Aqn6	中国	香港	
2020/12/21	3:28:02	5i7eF9BH8a980Aqn6	中国	山东	潍坊 联通
2020/12/21	4:25:07	5i7eF9BH8a980Aqn6	中国	北京	北京 电信
2020/12/21	7:15:06	5i7eF9BH8a980Aqn6	中国	香港	tencent.com
2020/12/21	8:06:02	wFhZg8327M2911mPSCYH	中国	广东	深圳 电信
2020/12/21	8:06:17	5i7eF9BH8a980Aqn6	中国	内蒙古	呼和浩特 联通